§ 8. Персистентное дерево отрезков

В § 2 дерево отрезков хранило единственное состояние массива. В § 7 мы научились сохранять старые корни и разделять неизменяемые узлы. Теперь соединим эти идеи. Одно точечное изменение затрагивает только путь от корня к листу, поэтому сохранение версии не требует копирования всего массива.

У этого приёма два разных применения. Первое — изменяемый массив с ветвящейся историей. Второе — статический массив, для каждого префикса которого строится частотное дерево по значениям. Во втором случае разность двух корней описывает диапазон позиций. Смешивать эти модели нельзя: одинаковый тип узла ещё не означает одинаковый смысл версии.

8.1. Контракт: какой массив представляет корень

Пусть длина $n > 0$ фиксирована. Интерфейс `set(root,pos,x)` возвращает новый корень с присвоенным значением в позиции `pos`; `sum(root,l,r)` возвращает сумму на полуинтервале $[l, r)$. Допускаются отрицательные значения и пустой запрос. Индексы принадлежат $[0, n)$, границы запроса удовлетворяют $0 \le l \le r \le n$. Все промежуточные суммы должны помещаться в `long long`.

Корень относится к конкретному экземпляру структуры и к одной фиксированной длине. Нельзя передавать корень другого пула, даже если его числовой номер существует в первом. Класс хранит узлы, а вызывающий код — нужные корни: эта граница ответственности позволяет оставить только интересующие версии.

Для начала используем нулевой массив. Корень 0 означает, что все значения равны нулю; его дети тоже имеют номер 0. Один служебный узел представляет любой полностью нулевой диапазон, потому что границы передаются отдельно. Это не полноценное дерево из $2n - 1$ узлов, а неявное представление. Оно особенно удобно для частот.

Редакторский пример. Для $n = 4$ создадим $r_1 = set(0, 1, 7)$, затем $r_2 = set(r_1, 3, 5)$ и независимо $r_3 = set(r_1, 1, -2)$. Получаются массивы $[0, 7, 0, 0]$, $[0, 7, 0, 5]$ и $[0, -2, 0, 0]$. Запрос $[1, 4)$ возвращает 7, 12 и −2. После создания r_3 корень r_2 по-прежнему возвращает 12.

Корень	Родитель	Изменение	Сумма всего массива
0	—	нулевой массив	0
r_1	0	позиция 1 получает 7	7
r_2	r_1	позиция 3 получает 5	12
r_3	r_1	позиция 1 получает −2	−2

Это пример полной персистентности: новая версия может продолжать любой старый корень. Хронологически последний созданный корень не обязан быть родителем следующего.

8.2. Копирование пути и два инварианта

Узел хранит сумму своего диапазона и номера детей. При присваивании сначала читаем старый узел в локальную переменную. Если это лист, заменяем значение только в локальной копии. Иначе рекурсивно получаем нового ребёнка со стороны изменяемой позиции, второго ребёнка оставляем прежним и пересчитываем сумму. Затем сохраняем новый узел в пуле.

(c) Aleksey Tolstikov

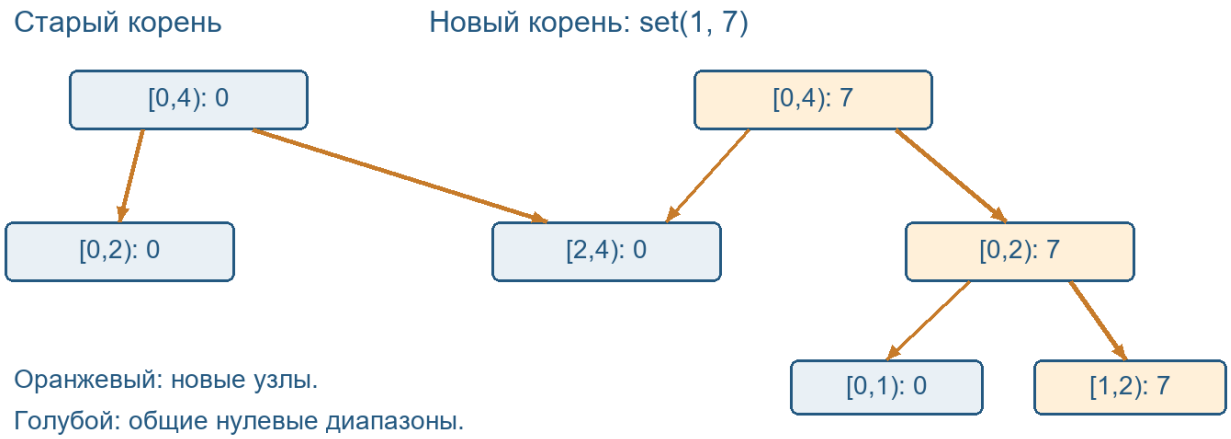


Рисунок 1: При изменении позиции 1 копируются три узла; правое поддерево используется обеими версиями.

Первый инвариант — обычный: сумма внутреннего узла равна сумме детей, а лист хранит значение соответствующей позиции. Второй — персистентный: после публикации корня ни один достижимый из него узел не изменяется. Недостаточно правильно пересчитать новый корень: нужно ещё доказать сохранность всех старых.

Доказательство идёт по длине диапазона. В листе новый узел содержит нужное значение, старый не тронут. Во внутреннем узле рекурсивный вызов по предположению индукции построил правильное изменённое поддерево; второе поддерево уже правильно и не меняется. Сумма их корней правильна. Новая родительская запись не меняет старые ссылки. Индукция доказывает оба утверждения одновременно.

Для $n = 2^h$ присваивание создаёт ровно $h + 1$ узлов, включая лист и корень. Для произвольного n — не более $\lceil \log_2 n \rceil + 1$. В нашей реализации даже присваивание прежнего значения создаёт путь: проверка равенства могла бы сэкономить память, но для оценки не требуется.

Корень 0 не надо раскрывать заранее. Копия служебного узла содержит нулевые ссылки; рекурсия создаст только нужную ветвь. Доказательство сохраняется, если считать отсутствующее поддерево полным нулевым массивом на переданном диапазоне.

8.3. Реализация: пул узлов и присваивание

Полный заголовок `code/08-persistent-segment.hpp` включает необходимые стандартные библиотеки. Ниже последовательные части одного класса. Пул `deque` позволяет добавлять узлы без переноса всех прежних записей; внешние ссылки всё равно представлены индексами. Локальная копия `Node t` исключает запись в общий узел.

(c) Aleksey Tolstikov

```
class PersistentSum {
public:
    using Root = std::size_t;
private:
    struct Node { long long sum=0; Root left=0, right=0; };
    std::deque<Node> pool{Node{}};
    int n;
    Root change(Root r, int lo, int hi, int pos, long long x) {
        Node t=pool[r];
        if (hi-lo==1) t.sum=x;
        else {
            int mid=lo+(hi-lo)/2;
            if (pos<mid) t.left=change(t.left,lo,mid,pos,x);
            else t.right=change(t.right,mid,hi,pos,x);
            t.sum=pool[t.left].sum+pool[t.right].sum;
        }
        pool.push_back(t);
        return pool.size()-1;
    }
```

Проверки `assert` выражают предусловия учебного интерфейса. Это не обработчик произвольного пользовательского ввода. В прикладном API ошибки индексов надо либо проверять явно, либо исключать условиями вызывающей программы. При `n=0` этот вариант класса не создаётся.

Почему новый узел добавляется после рекурсивного вызова? Тогда все его дети уже существуют. Вдоль каждой ссылки номер узла строго уменьшается, кроме служебного нуля. Следовательно, цикл в пуле невозможен. Номера задают топологический порядок графа представления, но не порядок версий: один корень может разделять узлы с несколькими историческими ветвями.

8.4. Чтение без выделения памяти

Запрос совпадает с обычным запросом суммы. При отсутствии пересечения возвращаем 0, при полном покрытии — готовую сумму, иначе складываем ответы детей. Нулевой корень тоже сразу возвращает 0. Запрос не создаёт и не меняет ни одного узла.

```
long long query(Root r,int lo,int hi,int l,int h) const {
        if (!r || h<=lo || hi<=l) return 0;
        if (l<=lo && hi<=h) return pool[r].sum;
        int mid=lo+(hi-lo)/2;
        return query(pool[r].left,lo,mid,l,h)
             + query(pool[r].right,mid,hi,l,h);
    }
```

Следующие методы завершают публичный интерфейс присваивания и суммы; позже к тому же классу добавим поиск порядковой статистики.

(c) Aleksey Tolstikov

```
public:
    explicit PersistentSum(int size): n(size) { assert(n>0); }
    Root set(Root r,int pos,long long x) {
        assert(r<pool.size() && 0<=pos && pos<n);
        return change(r,0,n,pos,x);
    }
    long long sum(Root r,int l,int h) const {
        assert(r<pool.size() && l>=0 && h>=0 && h<=n);
        assert(static_cast<unsigned>(l)<=static_cast<unsigned>(h));
        return query(r,0,n,l,h);
    }
    Root add(Root r,int pos,long long delta) {
        assert(0<=pos && pos<n);
        return set(r,pos,sum(r,pos,pos+1)+delta);
    }
```

Метод `add` читает старое значение и вызывает присваивание. Это два прохода высоты $O(\log n)$ и одно выделение нового пути; асимптотика остаётся прежней. При частых прибавлениях можно сделать один проход, но сначала полезнее иметь легко проверяемый контракт.

Для произвольного начального массива можно выполнить n присваиваний за $O(n \log n)$ времени и узлов. Это удобный, но не оптимальный способ. Обычное рекурсивное построение создаёт $2n - 1$ узлов за $O(n)$, после чего используется тот же алгоритм изменений. Для частотного дерева исходное состояние действительно нулевое, поэтому начальные $O(n)$ узлов вообще не нужны.

После q присваиваний от нулевого корня занято $O(1 + q \log n)$ узлов и $O(q)$ корней, если сохранять все версии. Запрос занимает $O(\log n)$ времени, включая пустой диапазон, и $O(\log n)$ стека рекурсии. Отдельный запрос ко всему массиву выполняется за $O(1)$.

8.5. Префиксные частоты: другая ось дерева

Теперь массив a статичен. Отсортируем различные значения и получим массив координат c. Значение a_i заменяется своим рангом в c. Сжатие сохраняет отношения порядка и равенства, но не расстояния: разность рангов нельзя считать разностью исходных значений.

Дерево строится уже не по позициям, а по рангам значений. Корень p_i хранит частоты первых i элементов: $a_0, \dots, a_{i-1}$. Поэтому $p_0 = 0$, а $p_{i+1} = add(p_i, rank(a_i), 1)$. В каждом листе число увеличивается на единицу при очередном появлении данного значения.

Для редакторского массива $a = [5, 1, 5, 3, 1]$ координаты равны $c = [1, 3, 5]$.

Префикс	Добавленное значение	Частоты значений 1, 3, 5
p_0	—	0, 0, 0
p_1	5	0, 0, 1
p_2	1	1, 0, 1
p_3	5	1, 0, 2
p_4	3	1, 1, 2
p_5	1	2, 1, 2

Диапазону позиций $[1, 5)$ соответствует разность $p_5 - p_1$: частоты $(2, 1, 1)$. Значения на этих позициях — $[1, 5, 3, 1]$. Мы не создаём физическое дерево разности: во время запроса читаем два дерева и вычитаем их агрегаты.

Для подсчёта элементов с $x \le a_i < y$ на позициях $[l, r)$ найдём $L = lower_bound(c, x)$ и $R = lower_bound(c, y)$. Ответ:

$$sum(p_r, L, R) - sum(p_l, L, R).$$

Пустой диапазон значений даёт ноль. Если нужна закрытая верхняя граница $a_i \le y$, для неё используется `upper_bound(c,y)`. Преобразование `y+1` опасно на максимальном целочисленном значении; двоичный поиск не требует этой арифметики.

Задача 3769 из плана курса относится к подсчёту в диапазонах позиций и значений. Это мотивация метода; приведённый интерфейс не подменяет точное условие iRunner, его индексацию и формат ввода.

8.6. K-я порядковая статистика по двум корням

Пусть $1 \le k \le r - l$. В левом ребёнке разности $p_r - p_l$ находится d элементов. Если $k \le d$, ответ лежит слева. Иначе пропускаем эти d элементов и ищем справа номер $k - d$. В листе получаем ранг, который затем переводится обратно в значение через c.

В нашем примере диапазон $[1, 5)$ в отсортированном виде равен $[1, 1, 3, 5]$. При разбиении трёх рангов на $[0, 1)$ и $[1, 3)$ слева находятся две единицы. Для $k = 3$ переходим вправо с $k = 1$, затем выбираем ранг значения 3. Повторы считаются отдельными элементами; заменять частоты булевыми флагами нельзя.

```
// Precondition: every leaf of after-before is nonnegative.
    int kth(Root before,Root after,long long k) const {
        assert(before<pool.size() && after<pool.size());
        long long total=pool[after].sum-pool[before].sum;
        if (k<1 || k>total) throw std::out_of_range("rank");
        int lo=0,hi=n;
        while (hi-lo>1) {
            int mid=lo+(hi-lo)/2;
            Root a=pool[before].left, b=pool[after].left;
            long long count=pool[b].sum-pool[a].sum;
            if (k<=count) { before=a; after=b; hi=mid; }
            else {
                k-=count;
                before=pool[before].right;
                after=pool[after].right;
                lo=mid;
            }
        }
        return lo;
    }
    std::size_t allocated() const { return pool.size()-1; }
};
```

Предусловие поиска сильнее, чем у суммы: частота в каждом листе разности должна быть неотрицательной. Проверка общего количества в коде этого не доказывает. Префиксная конструкция обеспечивает условие автоматически, когда $l \le r$. Для двух произвольных версий массива разность может содержать отрицательные листья, и такое ветвление уже не является поиском по мультимножеству.

Почему алгоритм корректен? Левый диапазон значений целиком меньше правого. Поэтому первые d элементов отсортированного мультимножества находятся слева, остальные справа. После выбора стороны меняется только локальный номер искомого элемента. Инвариант «ищем k-й элемент текущего мультимножества» сохраняется до листа.

Сжатие занимает $O(n \log n)$ времени, построение префиксов — $O(n \log m)$ времени и узлов, где m — число разных значений. Один поиск занимает $O(\log m)$, а не $O(\log n \log m)$: на каждом уровне читаются две пары детей. При $m = 1$ сразу возвращается единственное значение.

Полная сборка префиксов и два запроса

Следующая программа связывает сжатие координат с интерфейсом класса. Она хранится отдельно в `code/08-prefix-example.cpp`. Массив непустой; если исходные данные допускают пустой массив, нужно обработать его до создания дерева с размером 0. В учебном примере все координаты и номера версий помещаются в `int`.

```
#include "08-persistent-segment.hpp"
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<long long> a{5,1,5,3,1}, c=a;
    std::sort(c.begin(),c.end());
    c.erase(std::unique(c.begin(),c.end()),c.end());
    PersistentSum tree(static_cast<int>(c.size()));
    std::vector<PersistentSum::Root> prefix{0};
    for (long long x:a) {
        int rank=static_cast<int>(
            std::lower_bound(c.begin(),c.end(),x)-c.begin());
        prefix.push_back(tree.add(prefix.back(),rank,1));
    }
    int l=1,r=5;
    int low=static_cast<int>(
        std::lower_bound(c.begin(),c.end(),1)-c.begin());
    int high=static_cast<int>(
        std::lower_bound(c.begin(),c.end(),5)-c.begin());
    std::cout << tree.sum(prefix[r],low,high)
              -tree.sum(prefix[l],low,high) << '\n';
    int rank=tree.kth(prefix[l],prefix[r],3);
    std::cout << c[rank] << '\n';
}
```

Программа выводит две строки с числом 3. У первого ответа смысл «на позициях от 1 до 4 включительно три элемента принадлежат диапазону значений [1,5)». Это две единицы и тройка; пятёрка исключена верхней границей. У второго ответа смысл «третий элемент после сортировки выбранного диапазона равен 3». Совпадение чисел не означает совпадение операций.

Разберём типичную ошибку: если вместо `prefix[r]` взять `prefix[r-1]`, последний элемент диапазона потеряется. Тогда в первом ответе останутся только одна единица и тройка, получится 2. Если вместо `prefix[l]` вычесть `prefix[l-1]`, добавится элемент перед левой границей. В данном примере это пятёрка, и первый ответ случайно не изменится, поскольку верхняя граница её исключает. Поэтому одного численного запроса недостаточно для проверки индексации: нужны также запрос по всем значениям и поиск последнего элемента.

Полезный инвариант подготовки: `sum(prefix[i],0,m)==i` для каждого префикса. Он проверяет, что каждое добавление увеличило общее число ровно на один. Но он не проверяет распределение по рангам: перепутанные координаты могут сохранить сумму. Для полного теста сравнивают частоты отдельных значений, диапазонные количества и порядковые статистики с прямой сортировкой.

8.7. Какие агрегаты можно вычитать

Разность префиксов работает для аддитивных сведений. Частоты и суммы складываются по непересекающимся частям и допускают обратную операцию. Минимум двух префиксов вычитать бессмысленно: минимумы 1 и 1 не говорят, какие значения находятся между границами.

Если в частотных узлах дополнительно хранить сумму исходных значений, можно вычислять сумму k наименьших на диапазоне. При уходе вправо прибавляем сумму всего левого мультимножества, при уходе влево ничего не прибавляем. В листе добавляем $k \cdot c[rank]$. Счётчик определяет направление, сумма определяет вклад. Оба поля вычитаются между префиксами.

Например, для $[1, 5)$ сумма трёх наименьших равна $1 + 1 + 3 = 5$. При первом уходе вправо забираем сумму двух единиц, затем одну тройку. Отрицательные значения не мешают: порядок координат сохраняется, а частоты остаются неотрицательными. Нужно лишь контролировать переполнение суммы и произведения.

Задача 5003 о реконструкции стены в плане отмечена как углубление. Здесь мы выводим строительный блок «порядок плюс суммы», а не заявляем готовое решение задачи по одному её названию.

8.8. Почему изменение массива не меняет один префикс

Если заменить a_j в исходном статическом массиве, изменятся все частотные префиксы с индексами больше j. Нельзя обновить только p_{j+1} и оставить остальные: их содержимое перестанет соответствовать определению. Построенные префиксы — способ индексировать статические данные, а не бесплатная поддержка онлайн-изменений.

Для динамических запросов по позициям и значениям нужна дополнительная структура: например, дерево по позициям, в узлах которого хранится структура по значениям. Персистентность такого двухуровневого решения требует нового анализа времени и памяти. Оценку $O(\log n)$ для обычной суммы нельзя автоматически перенести на подсчёт по двум координатам.

С отложенными операциями возникает другая опасность. Обычный `push` из § 3 изменяет детей. Если эти дети общие со старой версией, их необходимо скопировать перед изменением. Чтение лучше организовывать без записи, передавая накопленную метку по рекурсии. Порядок композиции остаётся важным. Приведённый здесь код поддерживает только точечные изменения.

Связь с материалами. Копирование пути и ветвление разобраны в Lecture 11 SDS-en. Из видео 30.11.2022, 44:00–46:00, переносится вопрос о памяти на одну версию: для стека это один узел, для дерева — весь изменяемый путь. Этот фрагмент не выдаётся за объяснение частотных префиксов; числовые трассы главы разработаны для учебника.

8.9. Память, проверка и практические ошибки

При $n = 2^{20}$ одно присваивание создаёт 21 узел. Сто тысяч изменений требуют 2,1 миллиона узлов, даже если все они меняют одну позицию. В пуле с двумя `size_t` и `long long` полезная запись на типичной 64-битной платформе занимает 24 байта: около 50,4 МБ только записей, без служебных расходов контейнера и списка корней. Реальный размер проверяется через `sizeof(Node)` и измерение процесса.

В нашем пуле удаление корня не освобождает узлы. Это сознательная модель «храним всё до уничтожения экземпляра». Сборка мусора или подсчёт ссылок могут освобождать недостижимые узлы, но добавляют стоимость. Их нельзя молча включать в оценку времени `set`.

Для проверки версий используйте копии маленьких массивов. Каждый шаг выбирает случайный старый корень, создаёт изменение и затем сравнивает запросы в нескольких старых и новой версиях. Обязательно проверяйте длины 1, 3, 5, пустые запросы, отрицательные значения и повторные присваивания.

Для частот проверяйте каждый диапазон маленького массива прямой сортировкой. Сравнивайте все допустимые k, отдельно повторяющиеся значения и координаты вне диапазона данных. Проверяйте, что `sum` и `kth` не увеличивают счётчик выделенных узлов. Линейная цепочка версий не ловит все ошибки общей памяти: нужны братья в дереве истории.

8.10. Практика к главе

В расширенной подборке приложения A: №3769 «Персистентное дерево отрезков» (основная); №5003 «Реконструкция крепостной стены» (углубление). Полные условия доступны по ссылкам в приложении. Выберите основные задачи, затем переходите к углублению; решать весь список за одну неделю не требуется.

8.11. Вопросы для самопроверки

1. Чем отличается индекс в дереве версий массива от индекса префикса статического массива?
2. Какие узлы создаст присваивание позиции 5 при длине 8?
3. Почему один нулевой узел может представлять диапазоны разной длины?
4. Какие два утверждения нужно доказать при копировании пути?
5. Почему минимум на диапазоне нельзя получить вычитанием двух префиксных минимумов?
6. Какая проверка, кроме общего количества, нужна для корректности `kth` на произвольных корнях?
7. Как обработать закрытую верхнюю границу без вычисления `y+1`?
8. Почему замена одного элемента исходного массива меняет много частотных префиксов?
9. Какие сведения, кроме частот, нужны для суммы абсолютных отклонений от h на диапазоне? Почему планы задачи 5003 не создают обновлений массива?
10. Какой тест обнаружит изменение общего ребёнка через старую ссылку?

(c) Aleksey Tolstikov